

Module:4

Design of data path circuits

Binary Parallel Adder

- Add $A = 1011$ and $B = 0011$

Augend: A_i 1 0 1 1

Addend: B_i 0 0 1 1

Input carry: C_i 0 1 1 0

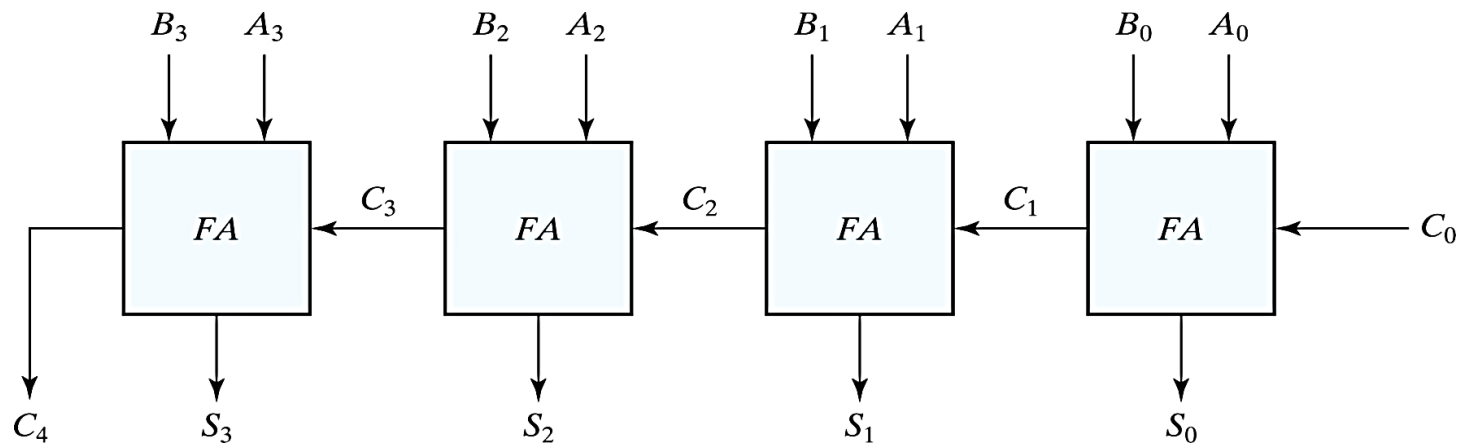
Sum: S_i 1 1 1 0

Carry: $Carry_{i+1}$ 0 0 1 1

subscript: i 3 2 1 0

Binary Parallel Adder / Ripple-Carry Adder

- Digital circuit that produces the arithmetic sum of two 'n' bit binary numbers, $n > 1$.
- It can be designed connecting the full adders in cascade with the carry of each full adder being connected to the input carry of the next full adder.
- Addition of 'n' bit requires 'n' full adders **or** (1 half adder and 'n-1' full adders)
- Called Parallel Adder because inputs are presented simultaneously (in parallel). Also, called **Ripple-Carry Adder**.



5-bit result

Binary Parallel Subtractor

- The subtraction $A - B$ can be done by taking the 2's complement of B and adding it to A because

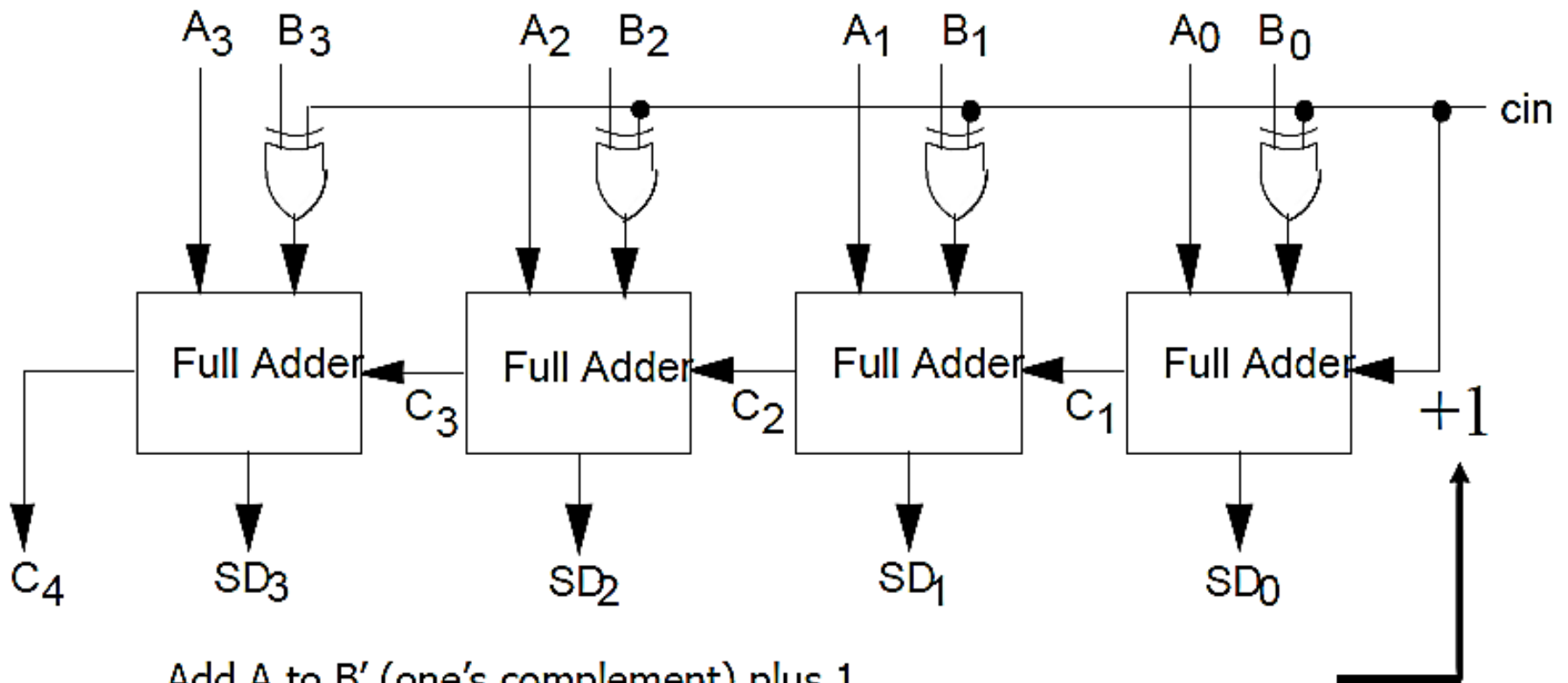
$$A - B = A + (-B) = A + ((B' + 1))$$

- It means if we use the inverters to make 1's complement of B (connecting each B_i to an inverter) and then add 1 to the least significant bit (by setting carry C_0 to 1) of binary adder, then we can make a binary subtractor.
- $1 \oplus B = B'.1 + B.1' = B'$

1's and 2's Complements

- 1's complement of 10111001 = 01000110
 - Simply replace 1's and 0's
- 1's complement of 10100010 = 01011101
- 2's complement of 10111001 = 01000110 + 1 = 01000111
 - Add 1 to 1's complement
- 2's complement of 10100010 = 01011101 + 1 = 01011110

4-bit Binary Subtractor



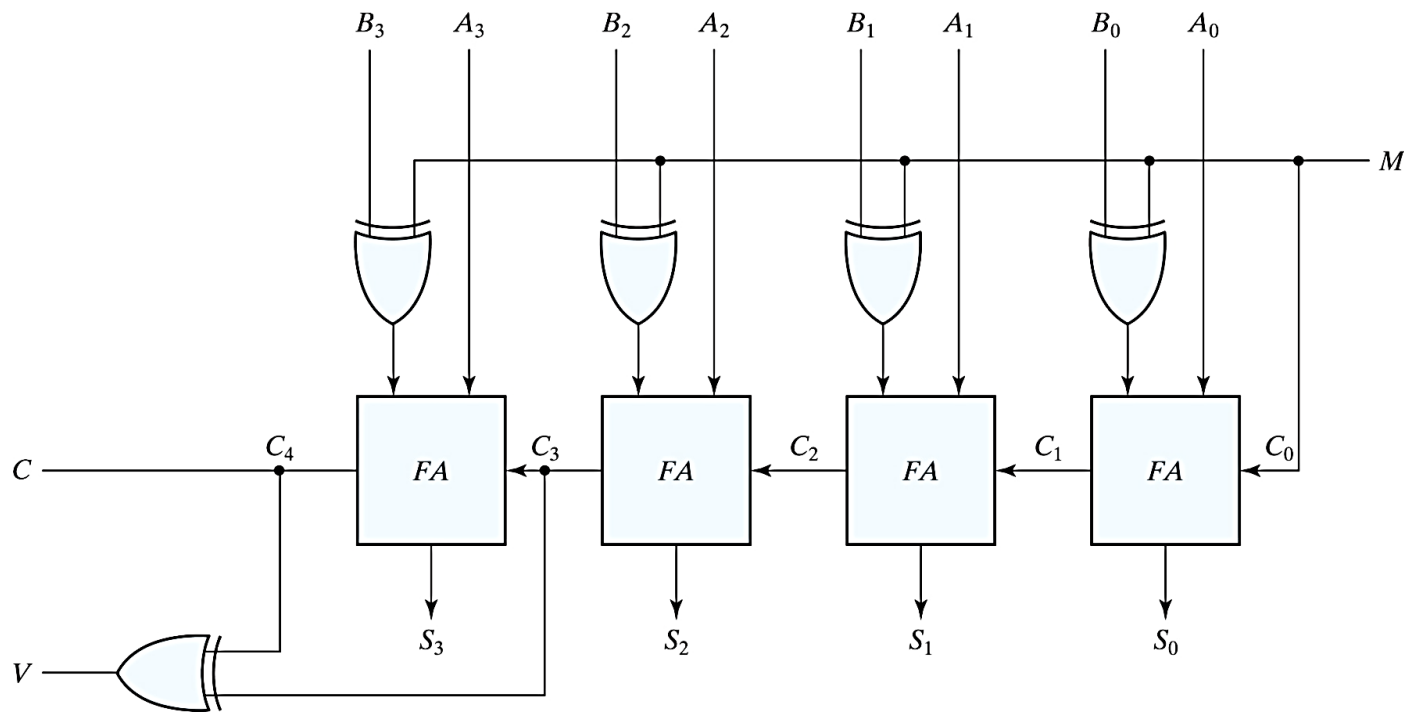
Add A to B' (one's complement) plus 1

That is, add A to two's complement of B

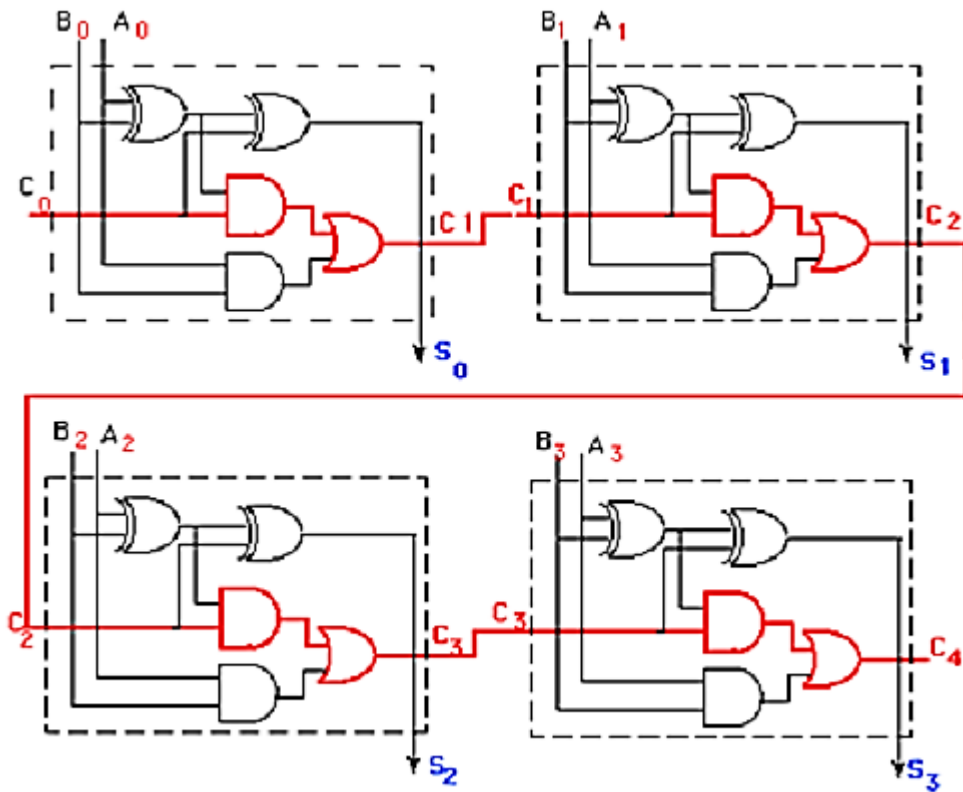
$$D = A - B$$

Binary Parallel Adder-Subtractor

- Addition and subtraction operation can be combined into one circuit with one binary adder and XOR gate with each full adder.
- The mode M controls the operation. When **M=0 the circuit is an Adder** when **M=1 the circuit is a Subtractor**.
- $0 \oplus B = B'.0 + B.0' = B$ and $1 \oplus B = B'.1 + B.1' = B'$



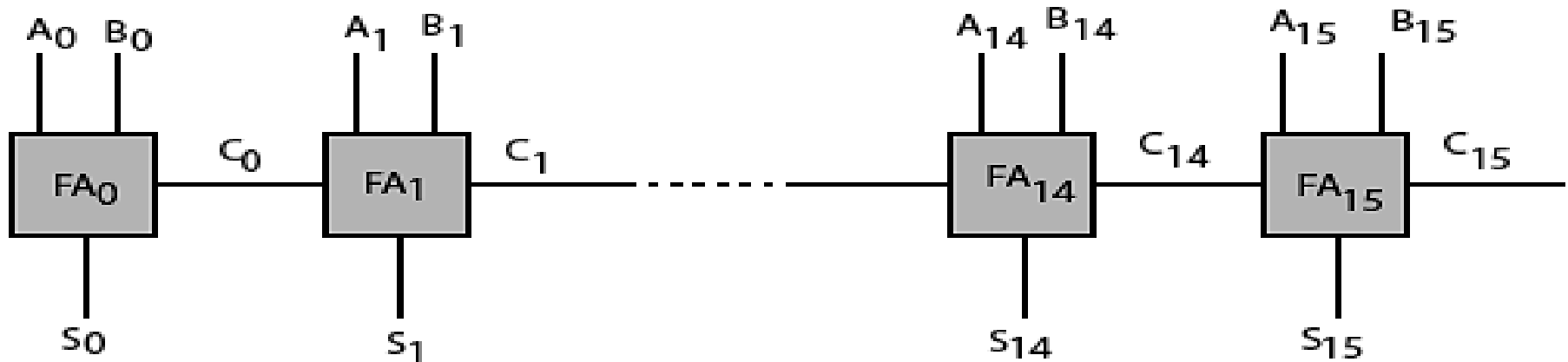
Parallel Adder/Ripple Adder Delay



- Assume gate delay = T
- $8T$ to compute the last carry = Carry Propagation
- Total delay = $8 + 1 = 9T$
- 1 delay from first half adder C_0
- Delay = $(2n+1)T$

It produces carry propagation delay while performing other arithmetic operations like multiplication and divisions as it uses several additions or subtraction steps.

A 16-bit ripple carry adder is realized using 16 identical full adders (FA) as shown in the figure. The carry-propagation delay of each FA is 12 ns and the sum-propagation delay of each FA is 15 ns. The worst-case delay (in ns) of this 16-bit adder will be ?



Solution:

Total number of full adders X Carry propagation delay of full adder

$$= 16 \times 12 \text{ ns} = 192 \text{ ns}$$

Time after which output sum bit becomes available from the last full adder

= Time taken for its carry in to become available + Sum propagation delay of full adder

= { Total number of full adders before last full adder X Carry propagation delay of full adder } + Sum propagation delay of full adder

$$= \{ 15 \times 12 \text{ ns} \} + 15 \text{ ns} = 195 \text{ ns}$$

Carry Look-Ahead Adder

- Reducing the carry propagation delay of adders is of great importance.
- One widely used approach to overcome the carry propagation problem is **carry look-ahead adder** which solves this problem by **calculating the carry signals in advance**, based on the input signals.
- In this design, the ripple carry design is suitably transformed such that the carry logic over fixed groups of bits of the adder is reduced to two-level logic.

Truth Table of Full Adder

Inputs			Outputs	
A	B	C	C+1	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$P_i = A_i \oplus B_i$$

$$G_i = A_i B_i$$

Truth Table of Full Adder

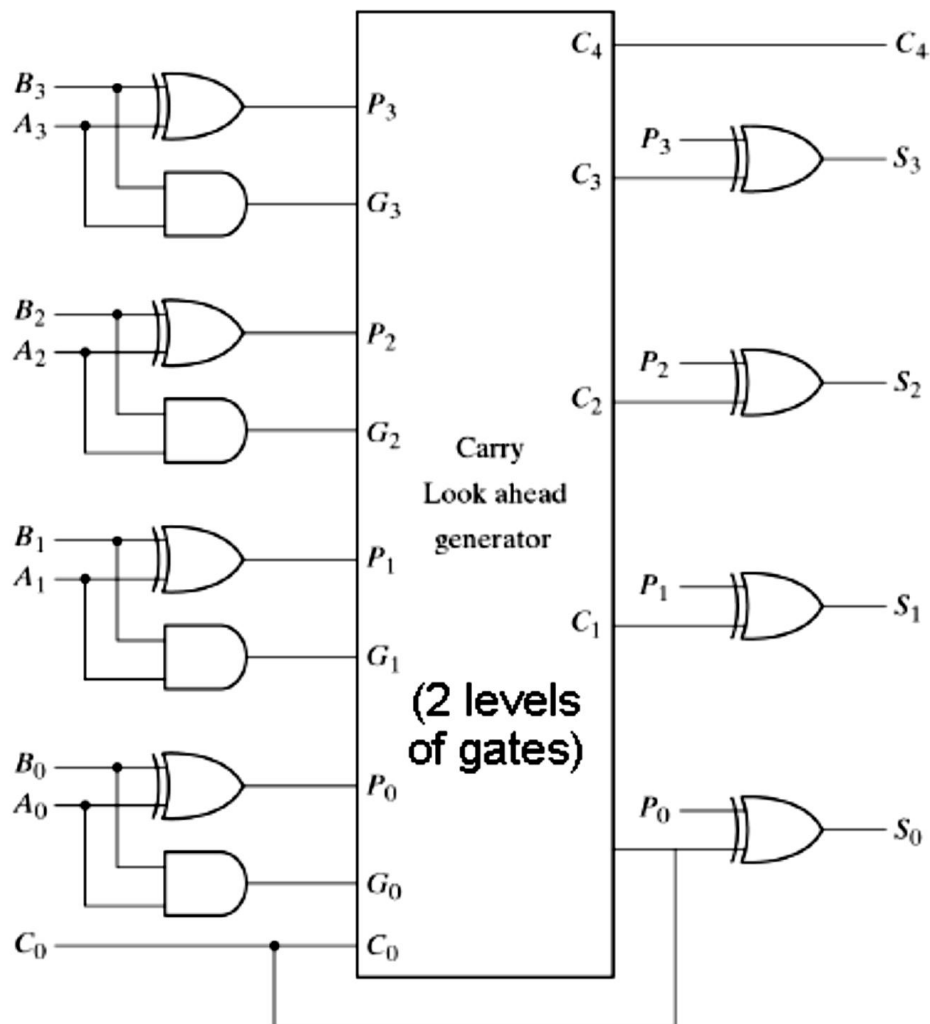
Inputs			Outputs
A	B	C	C+1
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Carry Propagate (Pi)

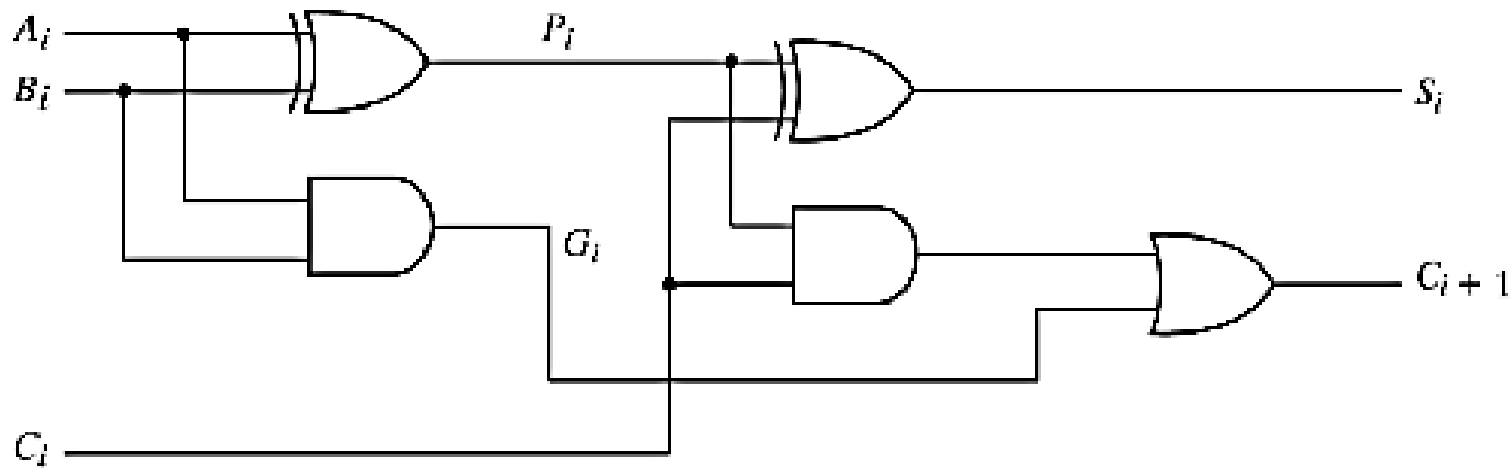
Carry Generate (Gi)

Generate: Both input bits (A and B) are 1

4-bit Carry Look-Ahead Adder



$$\begin{aligned}
 P_i &= A_i \oplus B_i \\
 G_i &= A_i B_i \\
 \text{sum } S_i &= P_i \oplus C_i \\
 \text{carry } C_{i+1} &= G_i + P_i C_i
 \end{aligned}$$



$$P_i = A_i \oplus B_i$$

$$G_i = A_i B_i$$

$$\text{sum } S_i = P_i \oplus C_i$$

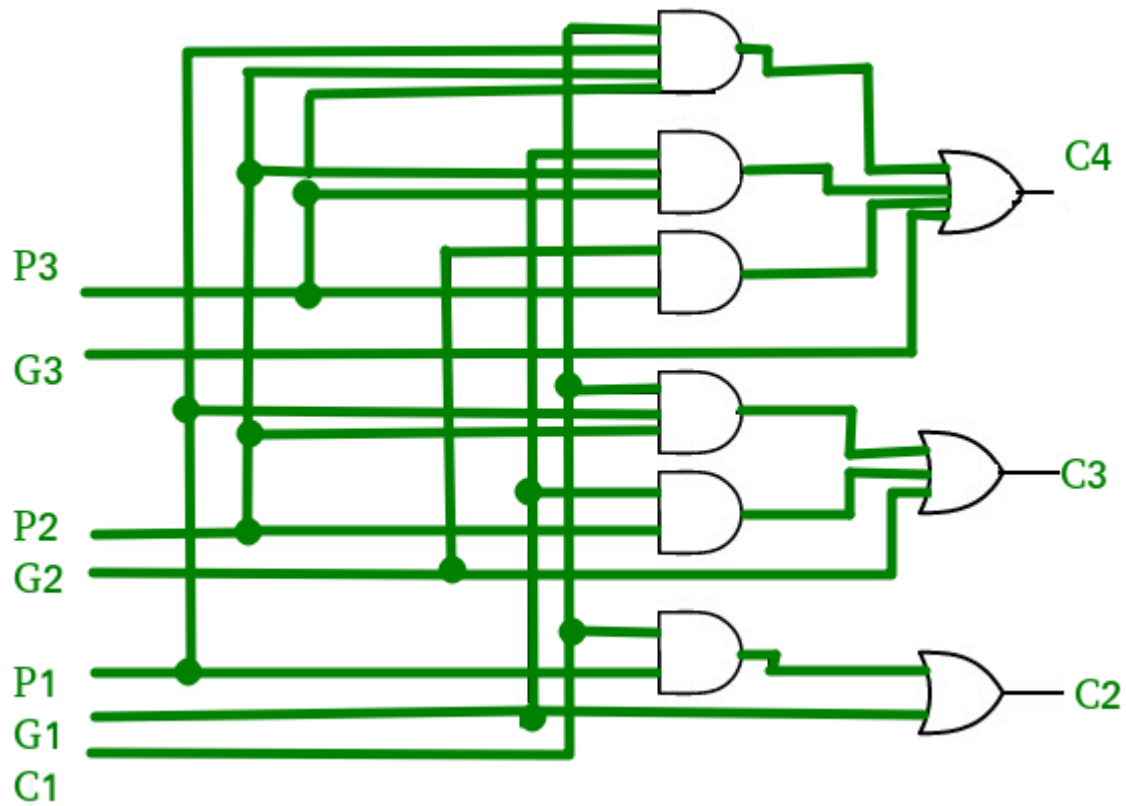
$$\text{carry } C_{i+1} = G_i + P_i C_i$$

$$C_1 = G_0 + P_0 C_{in}$$

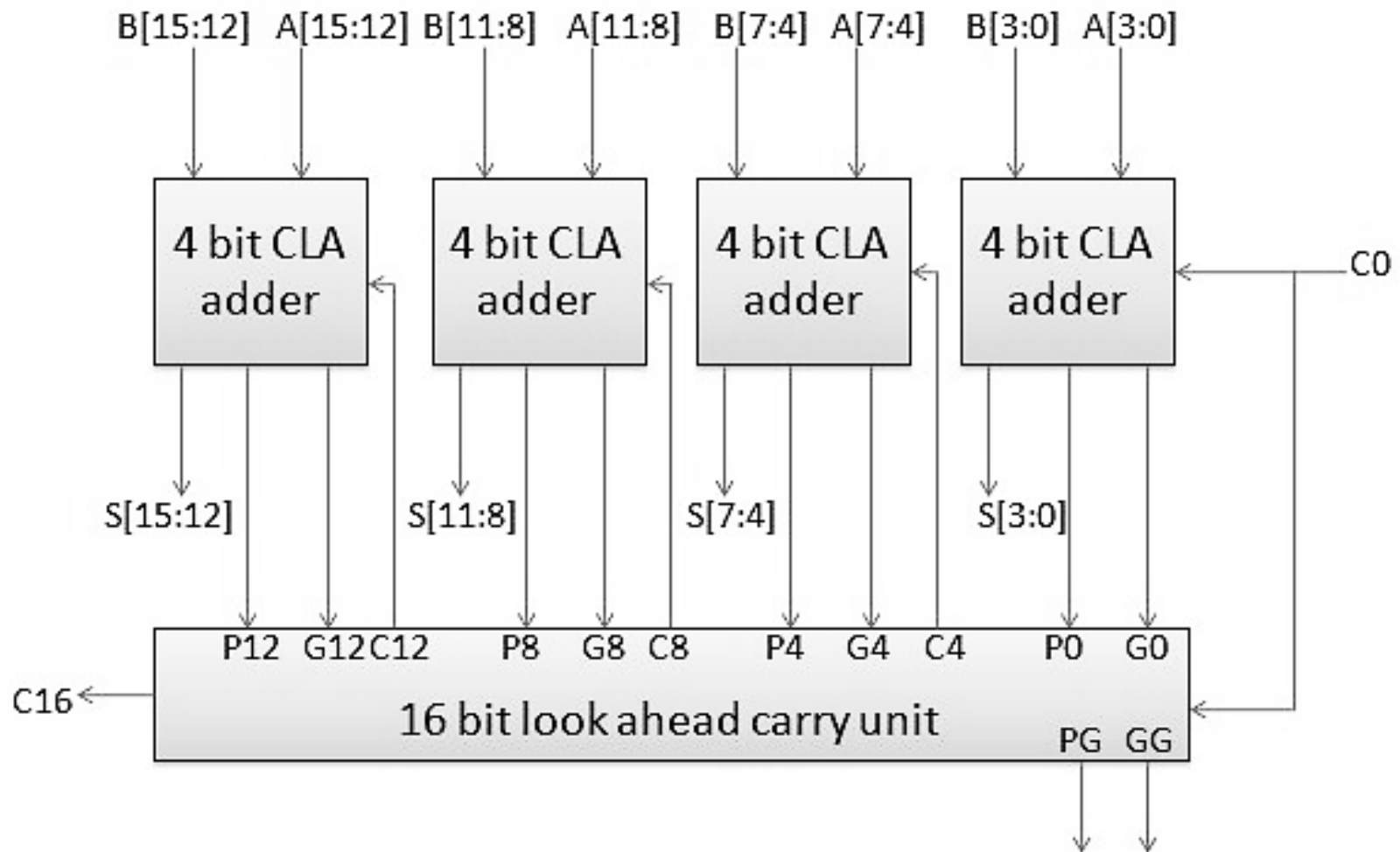
$$C_2 = G_1 + P_1 C_1 = G_1 + P_1 G_0 + P_1 P_0 C_{in}$$

$$C_3 = G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{in}$$

$$C_4 = G_3 + P_3 C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_{in}$$

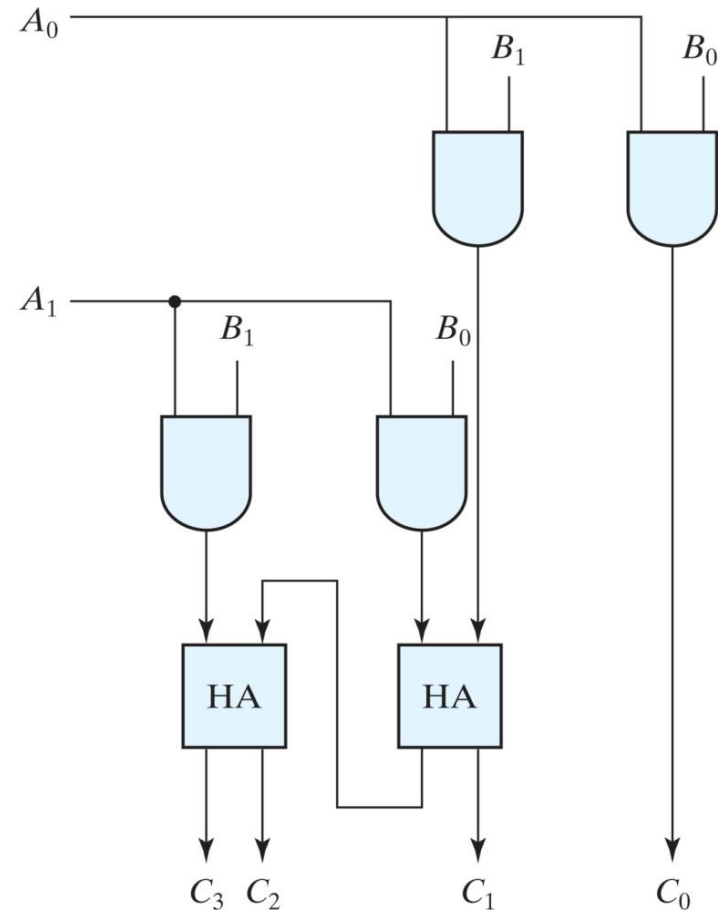


16-bit Carry Look-Ahead Adder Using 4-bit Carry Look-Ahead Adder



Multiplier – 2 bit by 2 bit

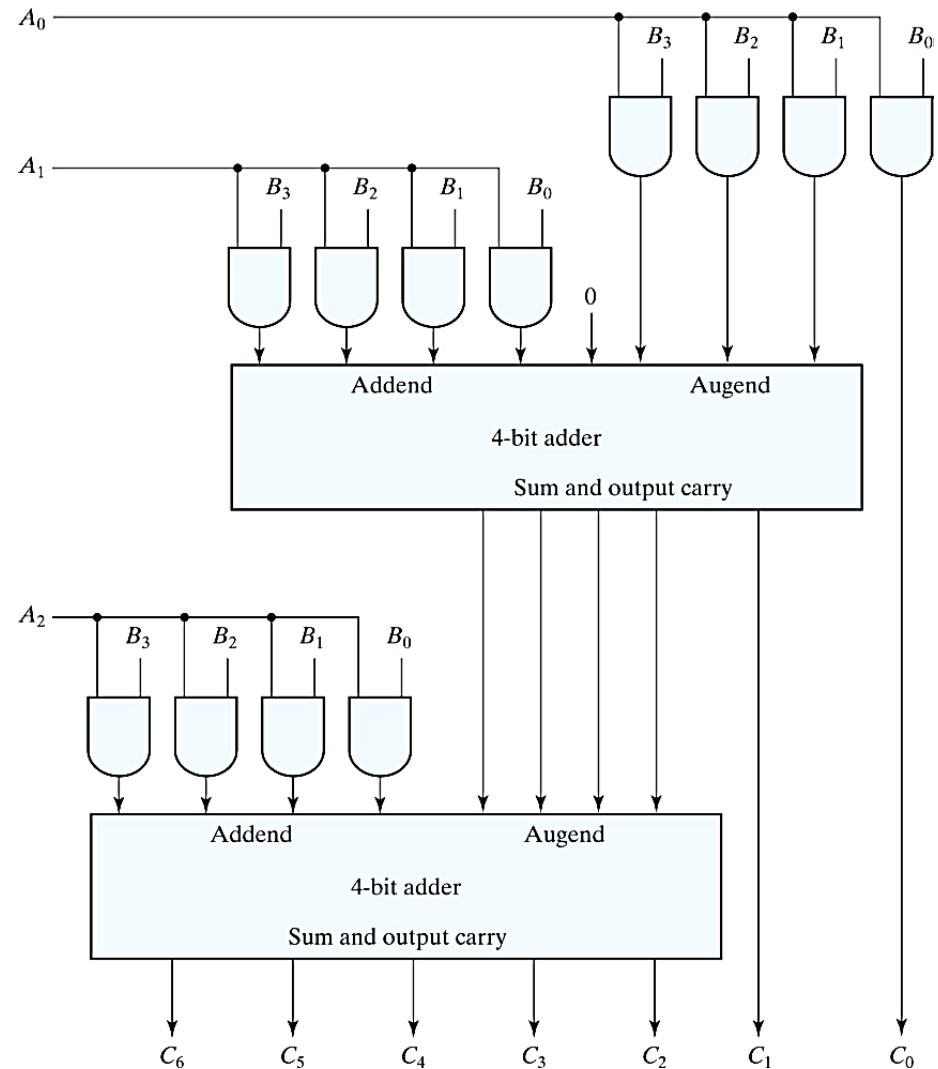
$$\begin{array}{r} B_1 \\ A_1 \\ \hline A_0 B_1 \\ A_1 B_0 \\ \hline C_3 C_2 C_1 \end{array}$$



Multiplier – 4 bit by 3 bit

$$\begin{array}{r}
 \begin{array}{cccc}
 B_3 & B_2 & B_1 & B_0 \\
 1 & 1 & 0 & 1 \\
 \times & 1 & 1 & 0 \\
 \hline
 0 & 0 & 0 & 0 & A_0 \\
 + & 1 & 1 & 0 & 1 & A_1 \\
 \hline
 1 & 1 & 0 & 1 & 0 \\
 + & 1 & 1 & 0 & 1 & A_2 \\
 \hline
 1 & 0 & 0 & 1 & 1 & 1 & 0
 \end{array}
 \end{array}$$

	B3	B2	B1	B0
A2				
A1				
A0				
0	A0B3	A0B2	A0B1	A0B0
0	A1B3	A1B2	A1B1	A1B0
A2B3	A2B2	A2B1	A2B0	
C ₆	C ₅	C ₄	C ₃	C ₂
C ₁	C ₀			

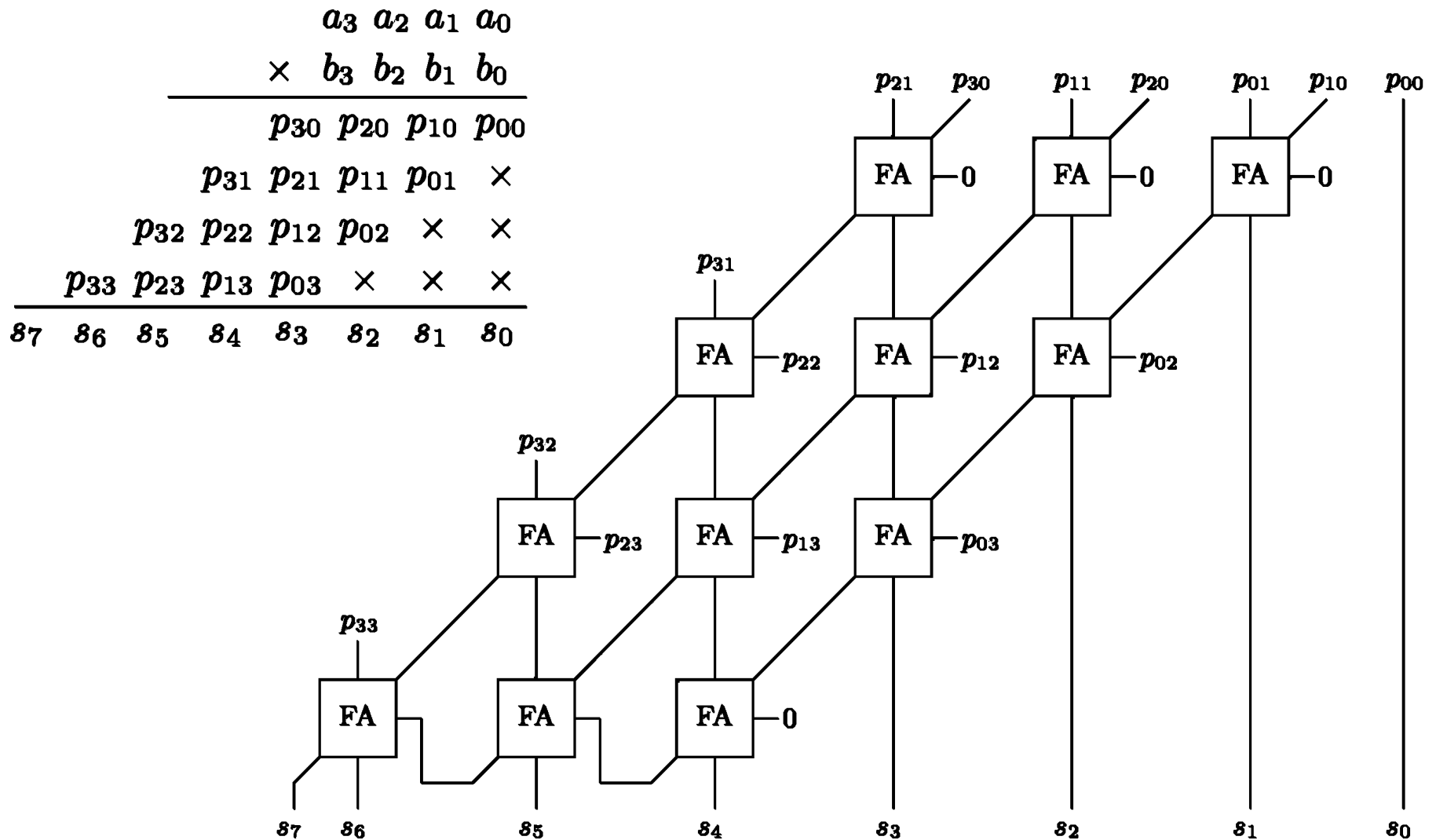


ARRAY MULTIPLIER

Array Multipliers

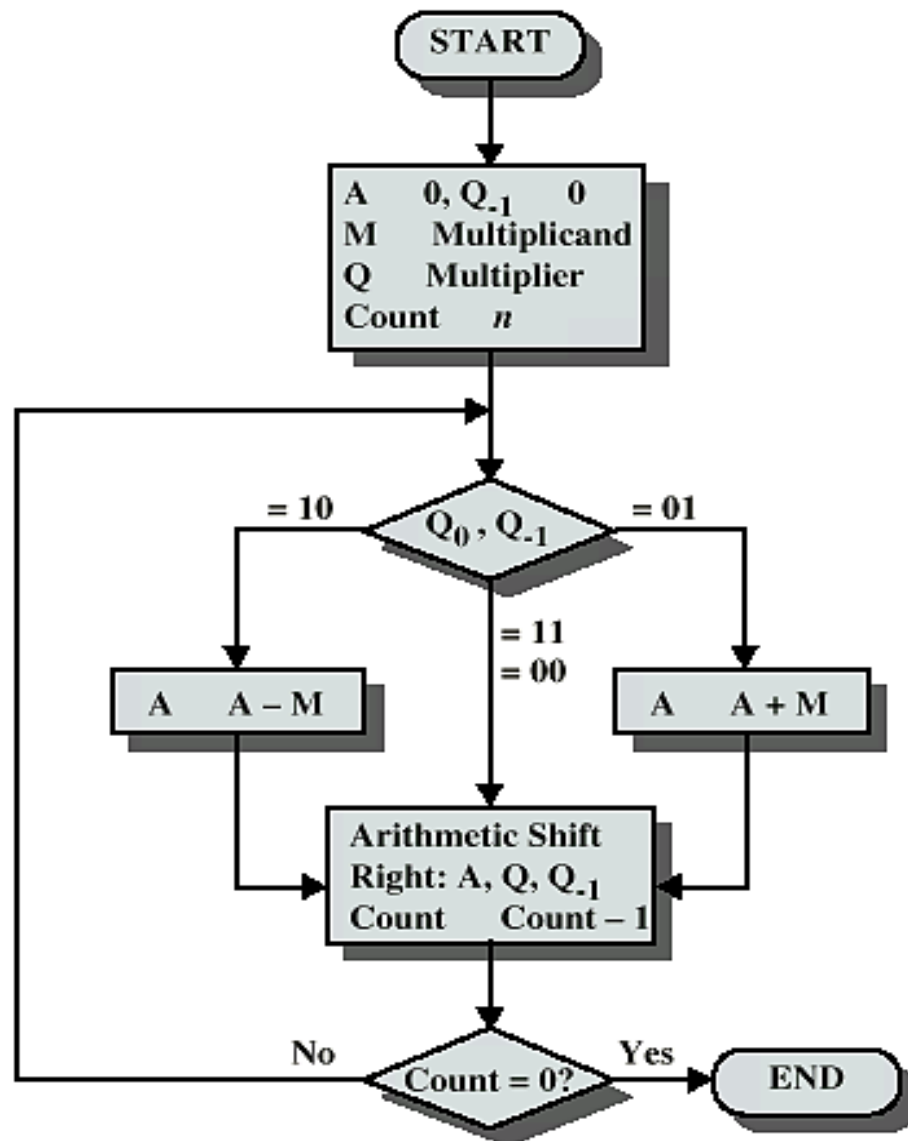
- The two basic operations - generation and summation of partial products - can be merged, avoiding overhead and speeding up multiplication
- **Iterative array multipliers** (or array multipliers) consist of identical cells, each forming a new partial product and adding it to previously accumulated partial product
 - Gain in speed obtained at expense of extra hardware
 - Can be implemented so as to support a high rate of pipelining

Circuit diagram



Booth's Algorithm

Booth's Algorithm



Booth Multiplier

Conditions:

1. If Q_0Q_{-1} are same i.e. **00** or **11** then, perform arithmetic **right shift** by 1 bit.
2. If $Q_0Q_{-1} = \mathbf{01}$ then perform
 $A \leftarrow A + M$
And then perform **arithmetic right shift**.
3. If $Q_0Q_{-1} = \mathbf{10}$ then perform
 $A \leftarrow A - M$
And then perform **arithmetic right shift**.

Multiplication 0100 x 0101 = (4x5)

M = 0100

Q = 0101

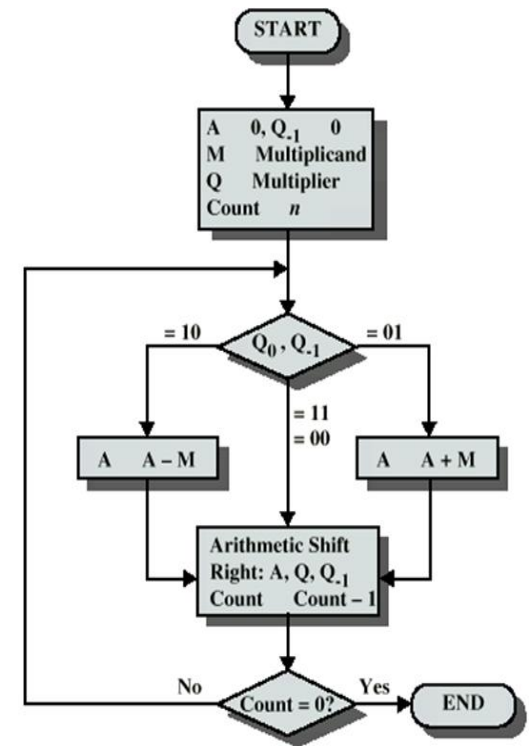
A = 0000

Q₋₁ = 0

Count = 4 (Number of bits)

Count	A				Q				Q ₋₁	Action
	A ₃	A ₂	A ₁	A ₀	Q ₃	Q ₂	Q ₁	Q ₀		
4	0	0	0	0	0	1	0	1	0	Check code of Q ₀ Q ₋₁
	1	1	0	0	0	1	0	1	0	A = A - M 0000 - 0100 = 0000 + 1100 (2's comp)
	1	1	1	0	0	0	1	0	1	ARS
3	1	1	1	0	0	0	1	0	1	Check code of Q ₀ Q ₋₁
	0	0	1	0	0	0	1	0	1	A = A + M 1110 0100+ 10010
	0	0	0	1	0	0	0	1	0	ARS
2	0	0	0	1	0	0	0	1	0	Check code of Q ₀ Q ₋₁
	1	1	0	1	0	0	0	1	0	A = A - M 0001 1100+ 1101
	1	1	1	0	1	0	0	0	1	ARS
1	1	1	1	0	1	0	0	0	1	Check code of Q ₀ Q ₋₁
	0	0	1	0	1	0	0	0	1	A = A + M 1110 0100+ 10010
	0	0	0	1	0	1	0	0	0	ARS
0	0	0	0	1	0	1	0	0		Stop

0100 x 0101 = 0001 0100



Multiplication 1100 x 0101 (-4 x 5 = -20)

M = 1100

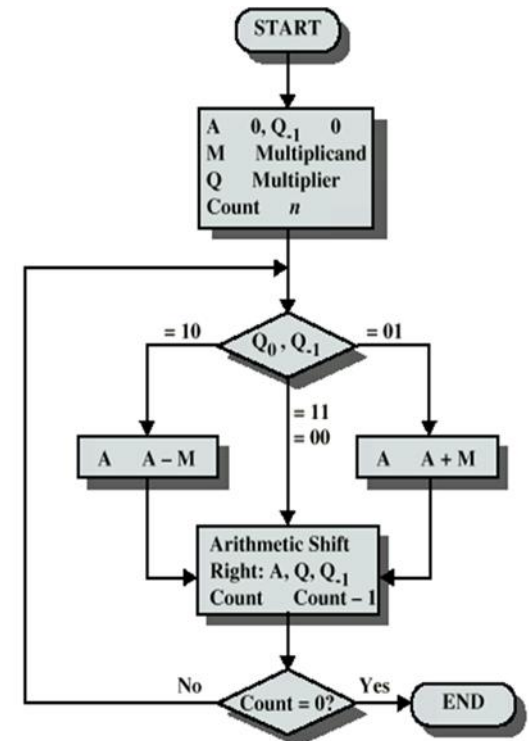
Q = 0101

A = 0000

Q₋₁ = 0

Count = 4 (Number of bits)

+4 = 0100
-4 = 1100
(2's Comp of 0100)



1100 x 0101 = 1110 1100

Count	A				Q				Q ₋₁	Action	
	A ₃	A ₂	A ₁	A ₀	Q ₃	Q ₂	Q ₁	Q ₀			
4	0	0	0	0	0	1	0	1	0	Check code of Q ₀ Q ₋₁	10
	0	1	0	0	0	1	0	1	0	A = A - M 0000 - 1100 = 0000 + 0100 (2's comp)	0000 0100+ 0100
	0	0	1	0	0	0	1	0	1	ARS	
3	0	0	1	0	0	0	1	0	1	Check code of Q ₀ Q ₋₁	01
	1	1	1	0	0	0	1	0	1	A = A + M	0010 1100+ 1110
	1	1	1	1	0	0	0	1	0	ARS	
2	1	1	1	1	0	0	0	1	0	Check code of Q ₀ Q ₋₁	10
	0	0	1	1	0	0	0	1	0	A = A - M	1111 0100+ 10011
	0	0	0	1	1	0	0	0	1	ARS	
1	0	0	0	1	1	0	0	0	1	Check code of Q ₀ Q ₋₁	01
	1	1	0	1	1	0	0	0	1	A = A + M	0001 1100+ 1101
	1	1	1	0	1	1	0	0	0	ARS	
0	1	1	1	0	1	1	0	0		Stop	

Booth : (7) x (3)

- Multiplicand (M) = 7 = 0111
- Multiplier (Q) = 3 = 0011

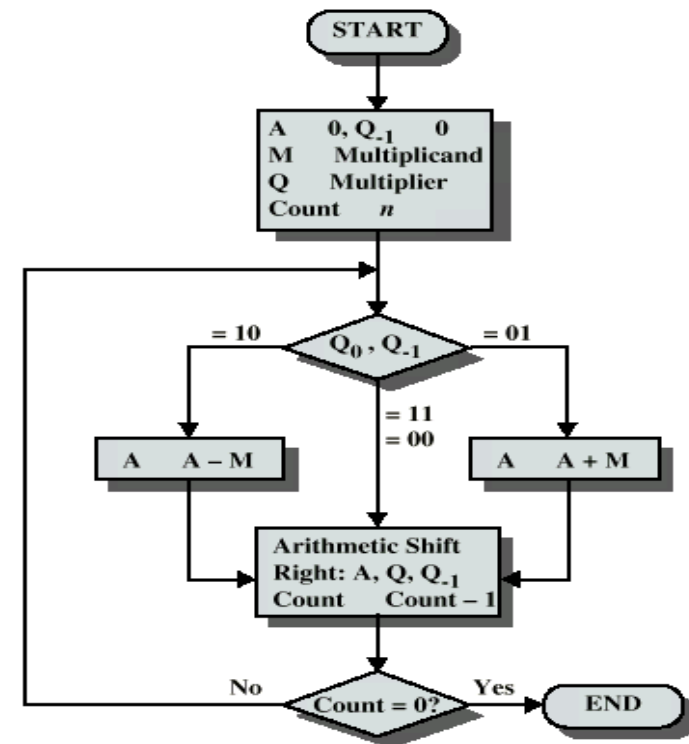
A	Q	Q-1	M	
	3		7	
0000	0011	0	0111	

1001	0011	0	0111	A ← (A - M) 1st
1100	1001	1	0111	Shift

1110	0100	1	0111	2nd Shift

0101	0100	1	0111	A ← (A + M) 3rd
0010	1010	0	0111	Shift

0001	0101	0	0111	4th Shift



Booth : (7) x (-3)

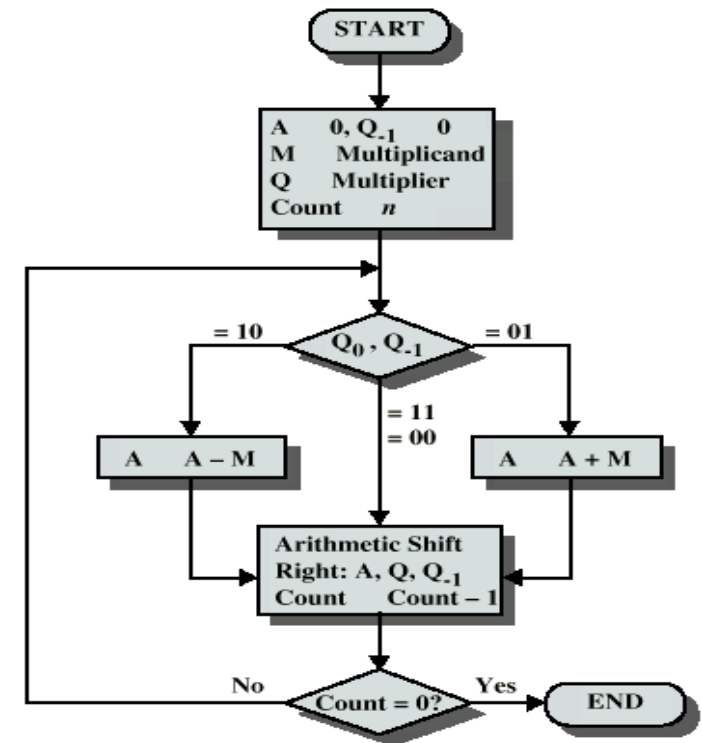
A	Q	M	
	-3	7	
0000	1101 0	0111	

1001	1101 0	0111	A ← (A - M) 1st
1100	1110 1	0111	Shift

0011	1110 1	0111	A ← (A + M) 2nd
0001	1111 0	0111	Shift

1010	1111 0	0111	A ← (A - M) 3rd
1101	0111 1	0111	Shift

1110	1011 1	0111	Shift 4th



Booth : (-7) x (3)

A	Q	M
	3	-7
0000	0011 0	1001

0111	0011 0	1001
0011	1001 1	1001

0001	1100 1	1001

1010	1100 1	1001
1101	0110 0	1001

1110	1011 0	1001

A ← (A - M) 1st
Shift

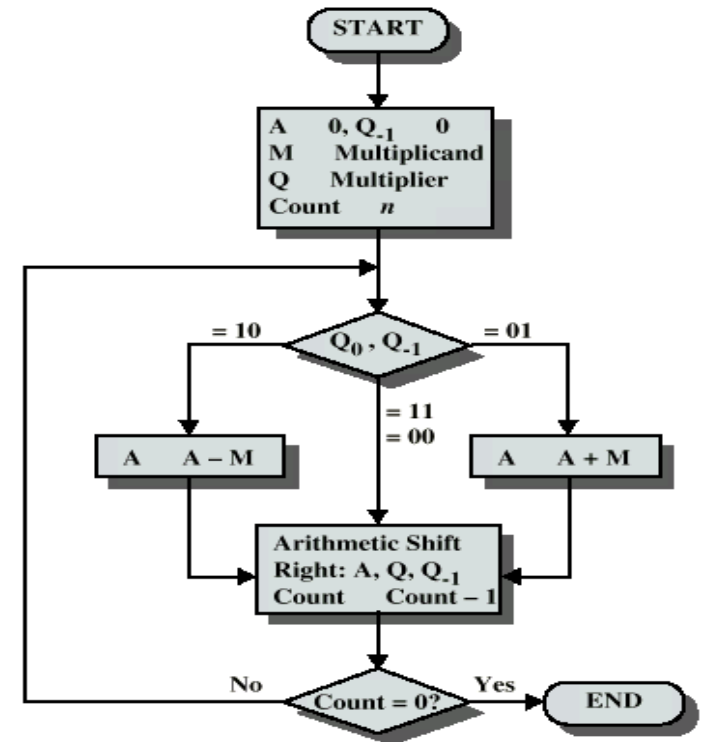
2nd

Shift

A ← (A + M) 3rd
Shift

4th

Shift



Booth : (-7) x (-3)

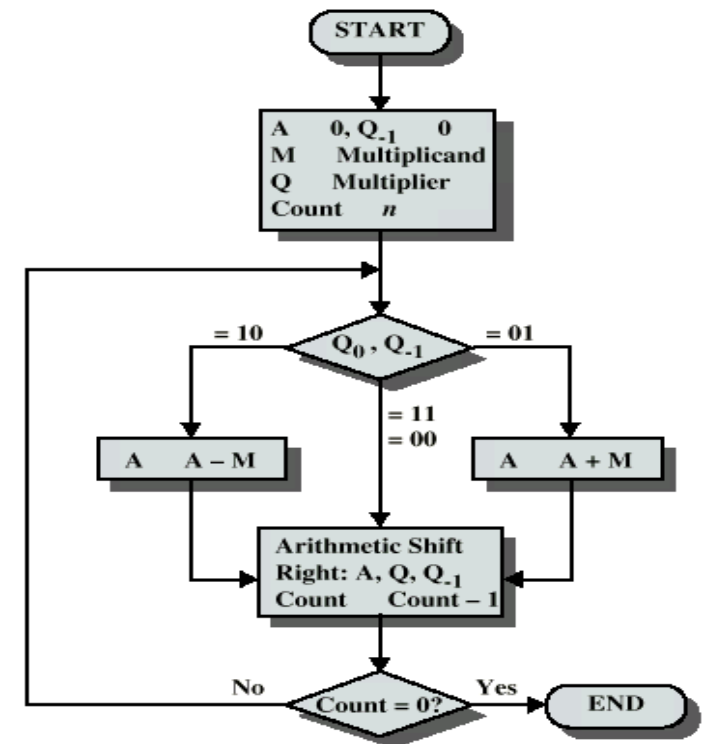
A	Q	M	
	-3	-7	
0000	1101 0	1001	

0111	1101 0	1001	A ← (A - M) 1st
0011	1110 1	1001	Shift

1100	1110 1	1001	A ← (A + M) 2nd
1110	0111 0	1001	Shift

0101	0111 0	1001	A ← (A - M) 3rd
0010	1011 1	1001	Shift

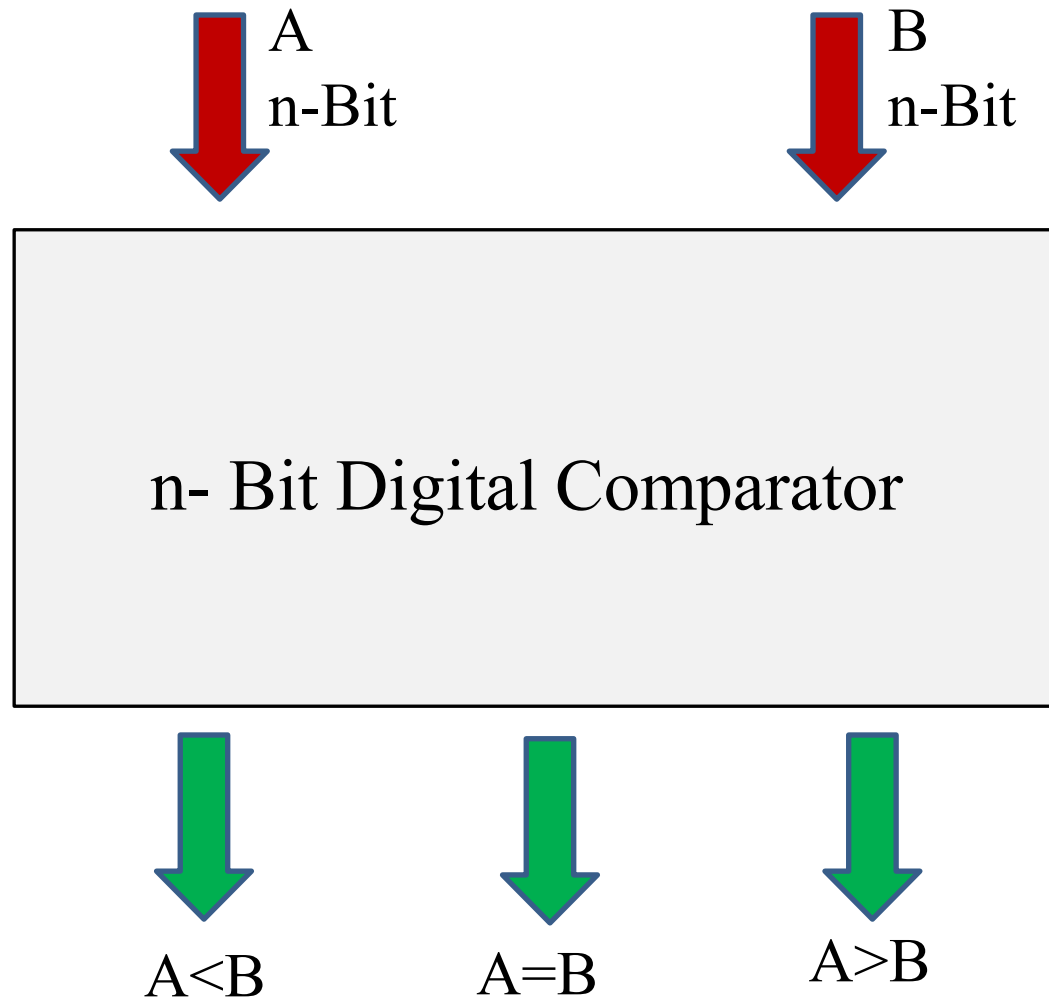
0001	0101 1	1001	Shift 4th



Magnitude Comparator: Digital Comparator

- It is a combinational logic circuit.
- Digital Comparator is used to compare the value of two binary digits.
- There are two types of digital comparator
 - (i) Identity Comparator
 - (ii) Magnitude Comparator.
- IDENTITY COMPARATOR: This comparator has only one output terminal for when $A=B$, either $A=B=1$ (High) or $A=B=0$ (Low)
- MAGNITUDE COMPARATOR: This Comparator has three output terminals namely $A>B$, $A=B$, $A<B$. Depending on the result of comparison, one of these output will be high (1)

Block Diagram of Magnitude Comparator



1- Bit Magnitude Comparator

- This magnitude comparator has two inputs A and B and three outputs $A < B$, $A = B$ and $A > B$.
- This magnitude comparator compares the two numbers of single bits.
- Truth Table of 1-Bit Comparator

INPUTS		OUTPUTS		
A	B	$Y_1 (A < B)$	$Y_2 (A = B)$	$Y_3 (A > B)$
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0

K-Maps For All Three Outputs :

B	$\overline{B}$	B
A	0	1
$\overline{A}$ 0	0	1
A 1	0	0

K-Map for $Y_1 : A < B$

$$Y_1 = \overline{A} B$$

B	$\overline{B}$	B
A	0	1
$\overline{A}$ 0	1	0
A 1	0	1

K-Map for $Y_2 : A = B$

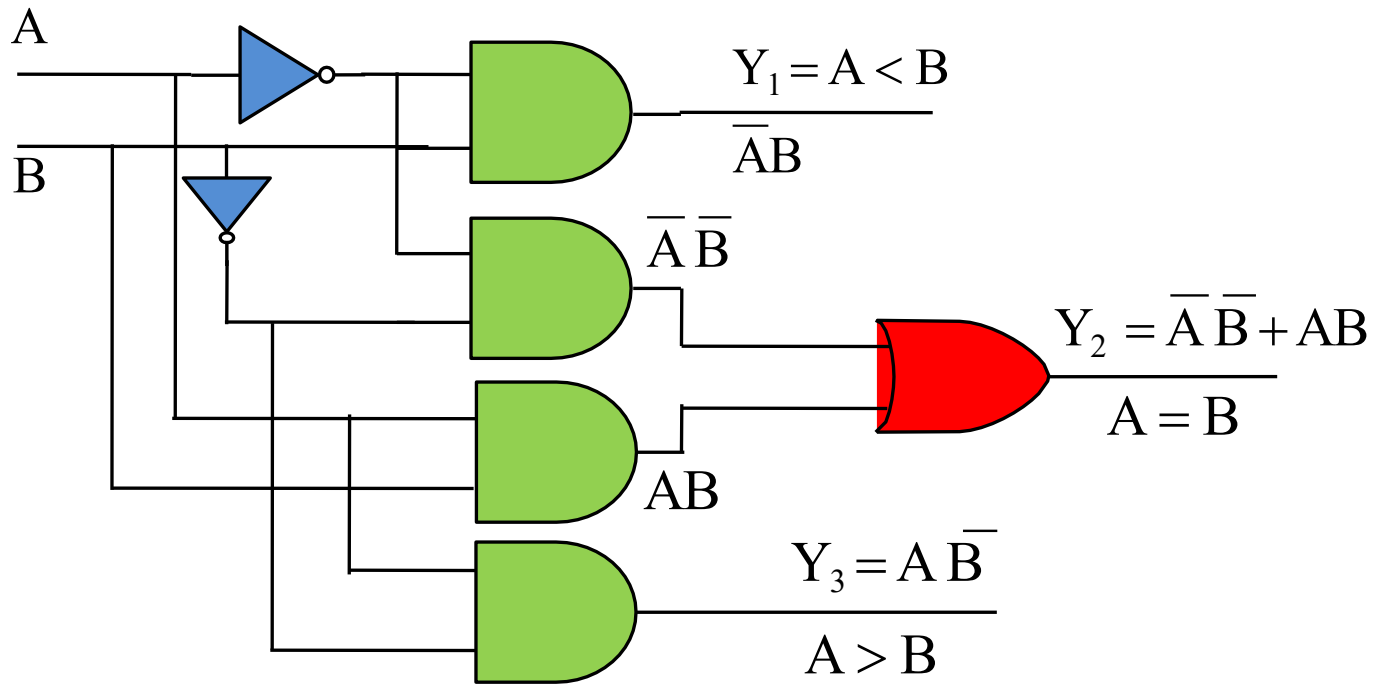
$$Y_2 = \overline{A}\overline{B} + AB$$

B	$\overline{B}$	B
A	0	1
$\overline{A}$ 0	0	0
A 1	1	0

K-Map for $Y_3 : A > B$

$$Y_3 = A\overline{B}$$

Realization of One Bit Comparator

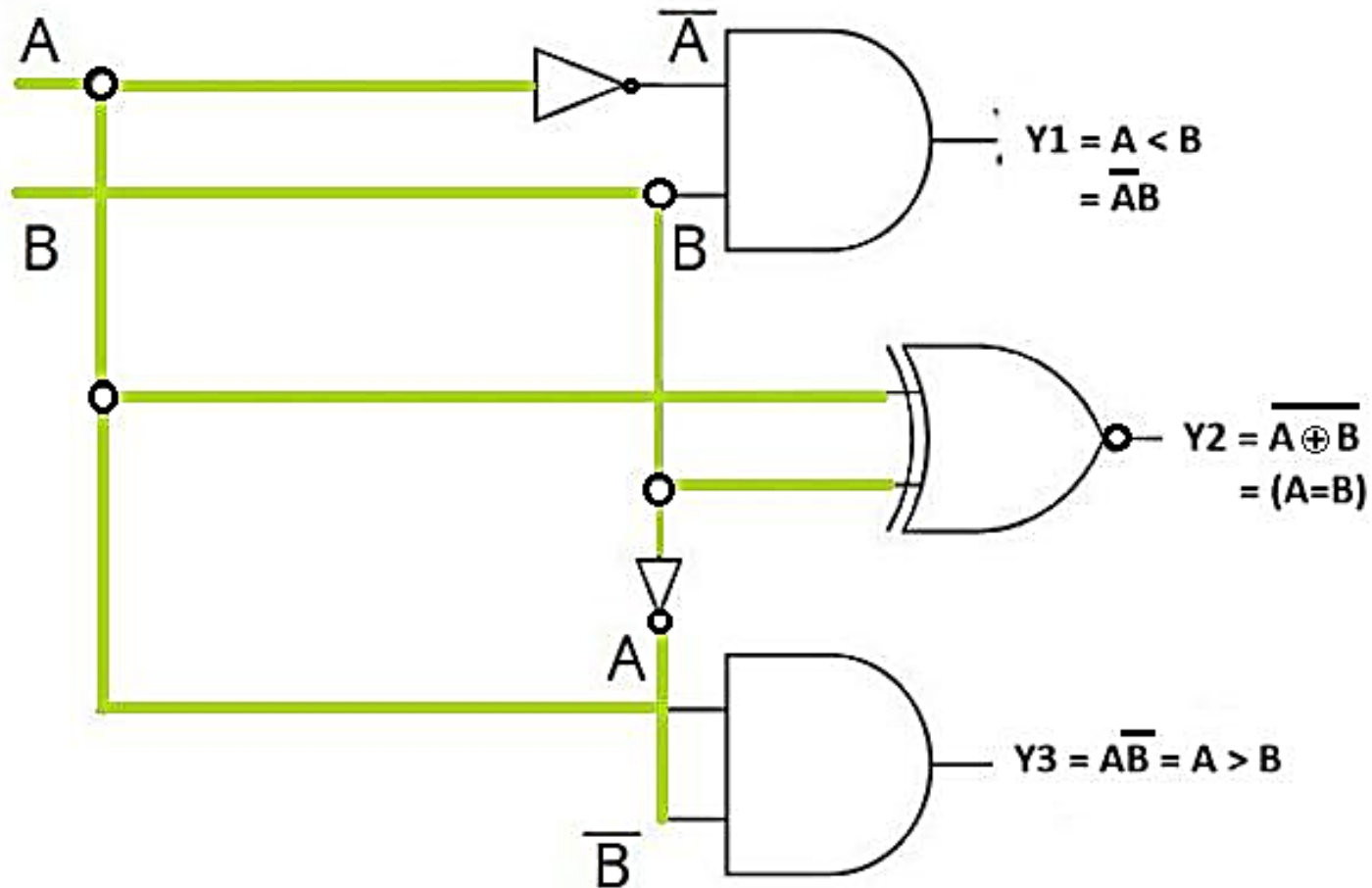


$$Y_1 = \overline{A}B$$

$$Y_2 = \overline{A}\overline{B} + AB$$

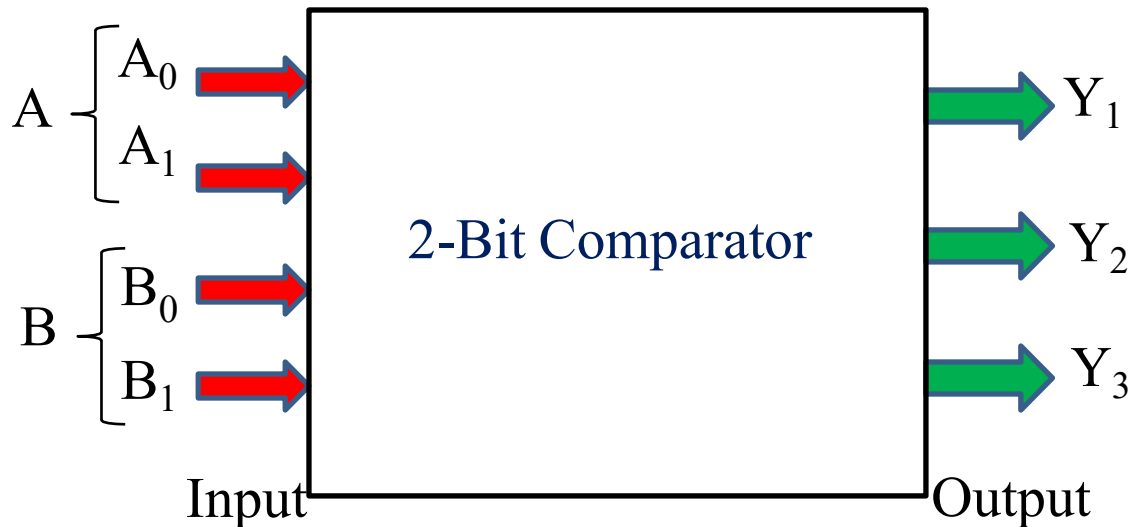
$$Y_3 = A\overline{B}$$

Realization Using AND , EX-NOR gates



2-Bit Comparator

- A comparator which is used to compare two binary numbers each of two bits is called a 2-bit magnitude comparator.
- It has four inputs and three outputs.
- Inputs are A_0 , A_1 , B_0 and B_1 and Outputs are Y_1 , Y_2 and Y_3



TRUTH TABLE

INPUT				OUTPUT		
A_1	A_0	B_1	B_0	$Y_1=A<B$	$Y_2=(A=B)$	$Y_3=A>B$
0	0	0	0	0	1	0
0	0	0	1	1	0	0
0	0	1	0	1	0	0
0	0	1	1	1	0	0
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	0	1	0
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	0	1	0

K-Map for A<B:

A ₁ A ₀ \ B ₁ B ₀				
	00	01	11	10
00	0	1	1	1
01	0	0	1	1
11	0	0	0	0
10	0	0	1	0

For A<B

$$Y_1 = \overline{A_1} \overline{A_0} B_0 + \overline{A_1} B_1 + \overline{A_0} B_1 B_0$$

K-Map for A=B:

A ₁ A ₀ \ B ₁ B ₀				
	00	01	11	10
00	1	0	0	0
01	0	1	0	0
11	0	0	1	0
10	0	0	0	1

For A=B

$$Y_2 = \overline{A_1} \overline{A_0} \overline{B_1} \overline{B_0} + \overline{A_1} A_0 \overline{B_1} B_0 + A_1 A_0 B_1 B_0 + A_1 \overline{A_0} B_1 \overline{B_0}$$

For A=B From K-Map

$$Y_2 = \overline{A_1} \overline{A_0} \overline{B_1} \overline{B_0} + \overline{A_1} A_0 \overline{B_1} B_0 + \overline{A_1} A_0 B_1 B_0 + A_1 \overline{A_0} B_1 \overline{B_0}$$

$$Y_2 = \overline{A_0} \overline{B_0} (\overline{A_1} B_1 + A_1 \overline{B_1}) + A_0 B_0 (\overline{A_1} B_1 + A_1 \overline{B_1})$$

$$Y_2 = (\overline{A_1} B_1 + A_1 \overline{B_1}) (\overline{A_0} \overline{B_0} + A_0 B_0)$$

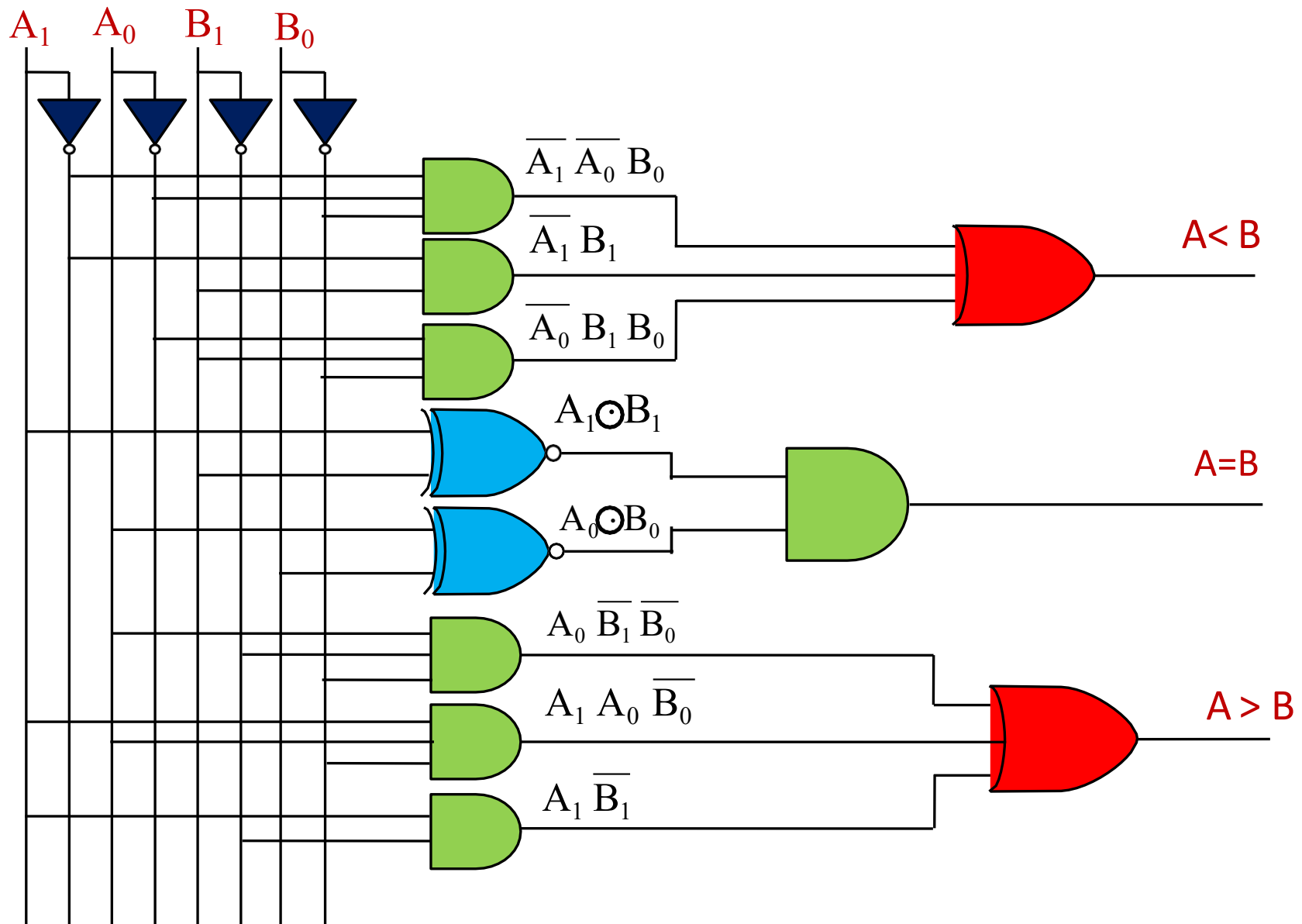
$$Y_2 = (A_1 \odot B_1) (A_0 \odot B_0)$$

K-Map For A>B

$A_1A_0 \backslash B_1B_0$		B_1B_0			
		00	01	11	10
00	0	0	0	0	0
01	1	0	0	0	0
11	1	1	0	1	0
10	1	1	0	0	0

$$Y_3 = A_0 \overline{B_1} \overline{B_0} + A_1 \overline{B_1} + A_1 A_0 \overline{B_0}$$

Logic Diagram of 2-bit Comparator:



4-bit Magnitude Comparison

- Algorithm -> logic
 - $A = A_3A_2A_1A_0$; $B = B_3B_2B_1B_0$
 - $A=B$ if $A_3=B_3, A_2=B_2, A_1=B_1$ and $A_0=B_0$
- Test each bit:
 - equality: $x_i = A_iB_i + A_i'B_i' = (A_iB_i' + A_i'B_i)'$
 - $(A=B) = x_3x_2x_1x_0$
- More difficult to test less than/greater than
 - $(A>B) = A_3B_3' + x_3A_2B_2' + x_3x_2A_1B_1' + x_3x_2x_1A_0B_0'$
 - $(A<B) = A_3'B_3 + x_3A_2'B_2 + x_3x_2A_1'B_1 + x_3x_2x_1A_0'B_0$
 - Start comparisons from **high-order** bits

A=B

- $A=B$ if
- $A_3=B_3, A_2=B_2, A_1=B_1$ and $A_0=B_0$
- $(A=B) = x_3x_2x_1x_0$
- $x_i = A_iB_i + A_i'B_i' = (A_iB_i' + A_i'B_i)'$

A > B

- For A > B
- $A_3 > B_3$,
- $A_3 = B_3$, $A_2 > B_2$,
- $A_3 = B_3$, $A_2 = B_2$, $A_1 > B_1$,
- $A_3 = B_3$, $A_2 = B_2$, $A_1 = B_1$, $A_0 > B_0$
- $(A > B) = A_3 B_3' + x_3 A_2 B_2' + x_3 x_2 A_1 B_1' + x_3 x_2 x_1 A_0 B_0'$

$$A < B$$

For $A < B$

$$B_3 > A_3,$$

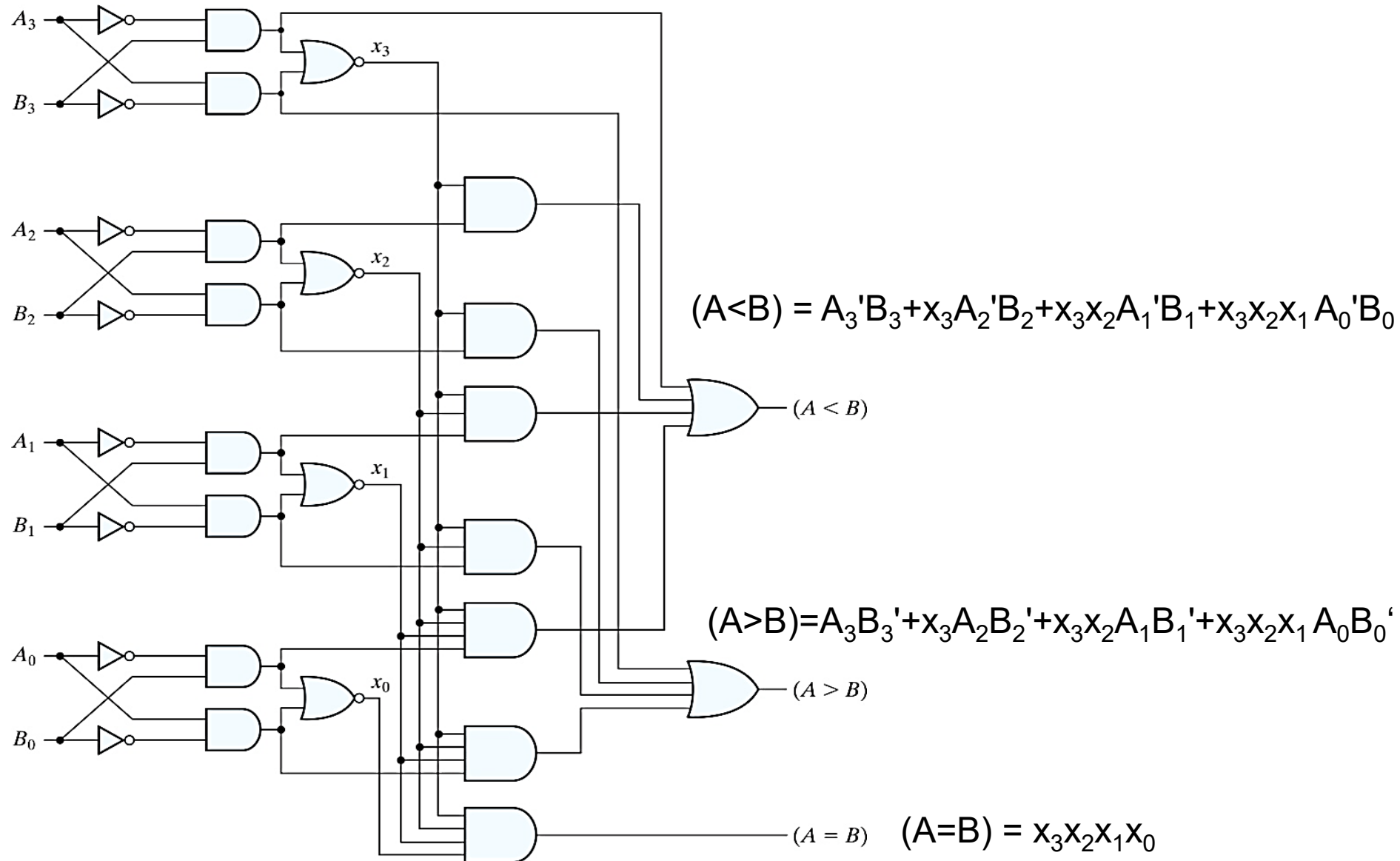
$$A_3 = B_3, B_2 > A_2,$$

$$A_3 = B_3, A_2 = B_2, B_1 > A_1,$$

$$A_3 = B_3, A_2 = B_2, A_1 = B_1, B_0 > A_0$$

$$(A < B) = A'_3 B_3 + x_3 A'_2 B_2 + x_3 x_2 A'_1 B_1 + x_3 x_2 x_1 A'_0 B_0$$

Logic Circuit for Magnitude Comparison



BCD Adder

- The value of A and B can vary from 0(0000 in binary) to 9(1001 in binary)
- When adding BCD numbers, if the sum of two BCD digits is greater than 9, the result is greater than 1001 in binary and hence is not valid in BCD.
- A correction needs to be performed by adding 0110 (6 in BCD) to the sum to get the correct BCD result.

BCD Adder

Decimal	Binary Sum					BCD Sum				
	C'	S3'	S2'	S1'	S0'	C	S3	S2	S1	S0
0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	1	0	0	0	0	1
2	0	0	0	1	0	0	0	0	1	0
3	0	0	0	1	1	0	0	0	1	1
4	0	0	1	0	0	0	0	1	0	0
5	0	0	1	0	1	0	0	1	0	1
6	0	0	1	1	0	0	0	1	1	0
7	0	0	1	1	1	0	0	1	1	1
8	0	1	0	0	0	0	1	0	0	0
9	0	1	0	0	1	0	1	0	0	1
10	0	1	0	1	0	1	0	0	0	0
11	0	1	0	1	1	1	0	0	0	1
12	0	1	1	0	0	1	0	0	1	0
13	0	1	1	0	1	1	0	0	1	1
14	0	1	1	1	0	1	0	1	0	0
15	0	1	1	1	1	1	0	1	0	1
16	1	0	0	0	0	1	0	1	1	0
17	1	0	0	0	1	1	0	1	1	1
18	1	0	0	1	0	1	1	0	0	0
19	1	0	0	1	1	1	1	0	0	1

Design of BCD Adder

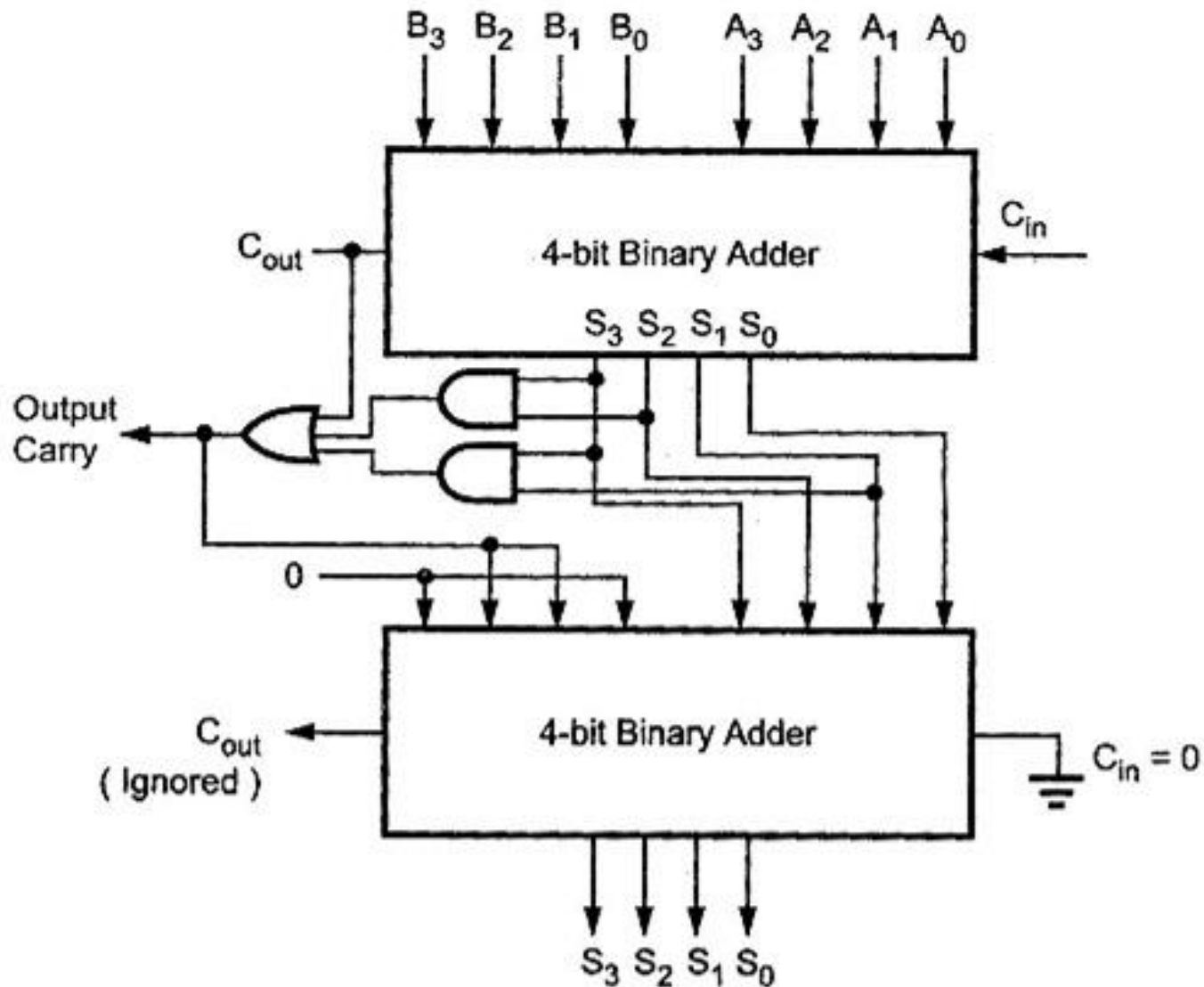
- To implement BCD Adder Circuit we require :
 - 4-bit binary adder for initial addition
 - Logic circuit to detect sum greater than 9 and
 - One more 4-bit adder to add 01102 in the sum if sum is greater than 9 or carry is 1.
- The logic circuit to detect sum greater than 9 can be determined by simplifying the boolean expression of BCD Adder Truth Table.

Inputs				Output
S_3	S_2	S_1	S_0	Y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

S_3S_2 \ S_1S_0					
		00	01	11	10
00		0	0	0	0
01		0	0	0	0
11		1	1	1	1
10		0	0	1	1

$$Y = S_3S_2 + S_3S_1$$

Design of BCD Adder



Verilog Blocking

- Blocking assignment statements are assigned using = and are **executed one after the other** in a procedural block.
- However, this will not prevent execution of statements that run in a parallel block.
- All statements are executed in parallel when simulation starts.

Verilog Non-Blocking

- Non-blocking assignment allows assignments to be scheduled without blocking the execution of following statements and is specified by a <= symbol.
- The RHS of every non-blocking statement of a particular time-step is captured, and moves onto the next statement.
- **The captured RHS value is assigned to the LHS variable only at the end of the time-step.**

Verilog Blocking

```
module tb;
  reg [7:0] a, b, d, e;
  initial begin
    a = 8'hDA;
    $display ("a=0x%0h b=0x%0h", a, b);
    b = 8'hF1;
    $display ("a=0x%0h b=0x%0h", a, b);
  end
  initial begin
    d = 8'hAA;
    $display ("d=0x%0h e=0x%0h", d, e);
    e = 8'h55;
    $display ("d=0x%0h e=0x%0h", d, e);
  end
endmodule
```

Ans

a=0xDA b=0xx
a=0xDA b=0xF1
d=0xAA e=0xx
d=0xAA e=0x55

x = Hex
o = Octal
b = Binary

Verilog Non-Blocking

```
module tb;
  reg [7:0] a, b, d, e;
  initial begin
    a <= 8'hDA;
    $display ("a=0x%0h b=0x%0h", a, b);
    b <= 8'hF1;
    $display ("a=0x%0h b=0x%0h", a, b);
  end
  initial begin
    d <= 8'hAA;
    $display ("d=0x%0h e=0x%0h", d, e);
    e <= 8'h55;
    $display ("d=0x%0h e=0x%0h", d, e);
  end
endmodule
```

The captured RHS value is assigned to the LHS variable only at the end of the **time-step**.

Ans

a=0xx b=0xx

a=0xx b=0xx

d=0xx e=0xx

d=0xx e=0xx